SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Department of Artificial Intelligence & Machine Learning
Course: F0003 CA2/CA3 · Advanced Agentic AI

SYSTEM ARCHITECTURE & DESIGN SPECIFICATION

Multi-Agent Research & Report Factory

Technical Blueprint: 5-Agent Coordination, Typed Pub/Sub Bus & Non-Linear Epistemic Feedback Loops

PROJECT & TEAM SPECIFICATIONS

Repository: https://github.com/AnushkaTandon26/Multi-Agent-Reseach-and-Report-Factory

Live Demo: https://insight-foundry-three.vercel.app/

Track: CA3 Multi-Agent AI System (Evaluator-Optimizer Architecture)

Team Members:

- Anushka Tandon (23070122262) — Multi-Agent Architecture & Orchestrator (Lead)
- Jyoti Kumari Sah (23070122278) — Epistemic Verification & Critic Engine
- Muskan Shah (23070122279) — Literature Scout & Vector Retrieval Tools
- Lauriane Lenge Wa Mpitshi (23070122260) — Reader Agent & Data Pipeline

Submission Date: October 2026 | Document Version: 4.0 Final

TABLE OF CONTENTS

1. Architectural Blueprint & Orchestration Patterns 3
- 1.1 System Architecture Topology & Message Bus Fabric
- 1.2 Message Bus Architecture Diagram (Figure 6)
- 1.3 Message Protocol & Event Type Schema

2. Detailed Agent Specifications & Tool Harnesses 6
- 2.1 Station 01: Strategic Planner (PlannerAgent)
- 2.2 Station 02: Literature Scout (SearcherAgent)
- 2.3 Station 03: Deep Reader (ReaderAgent)
- 2.4 Station 04: Epistemic Critic (CriticAgent)
- 2.5 Station 05: Executive Synthesizer (WriterAgent)

3. Orchestrator Design & State Machine 10
- 3.1 Orchestrator State Transition Logic
- 3.2 Feedback Loop Implementation Details
- 3.3 Shared Memory Architecture

4. Data Flow & Pipeline Sequence 12
- 4.1 End-to-End Data Flow Diagram (Figure 4)
- 4.2 Input/Output Contracts per Agent

5. Technical Stack & Engineering Rationale 14
- 5.1 Technology Stack Diagram (Figure 3)
- 5.2 Technology Decisions & Trade-Off Analysis

6. UI Component Architecture & Theme System 15
- 6.1 Component Hierarchy & Responsibilities
- 6.2 Theme Token System Diagram (Figure 5)

7. Security, Error Boundaries & Fault Tolerance 17
- 7.1 Memory Isolation & Prompt Injection Defense
- 7.2 Graceful Degradation Patterns

8. Testing Architecture & CI/CD 18
- 8.1 Test Strategy & Coverage Matrix
- 8.2 Continuous Integration Pipeline

9. Performance Engineering & Optimization 19
- 9.1 Latency Optimization Strategies
- 9.2 Resource Utilization Benchmarks

Appendix: API Contracts, Type Definitions & Telemetry 20

1. ARCHITECTURAL BLUEPRINT & ORCHESTRATION PATTERNS

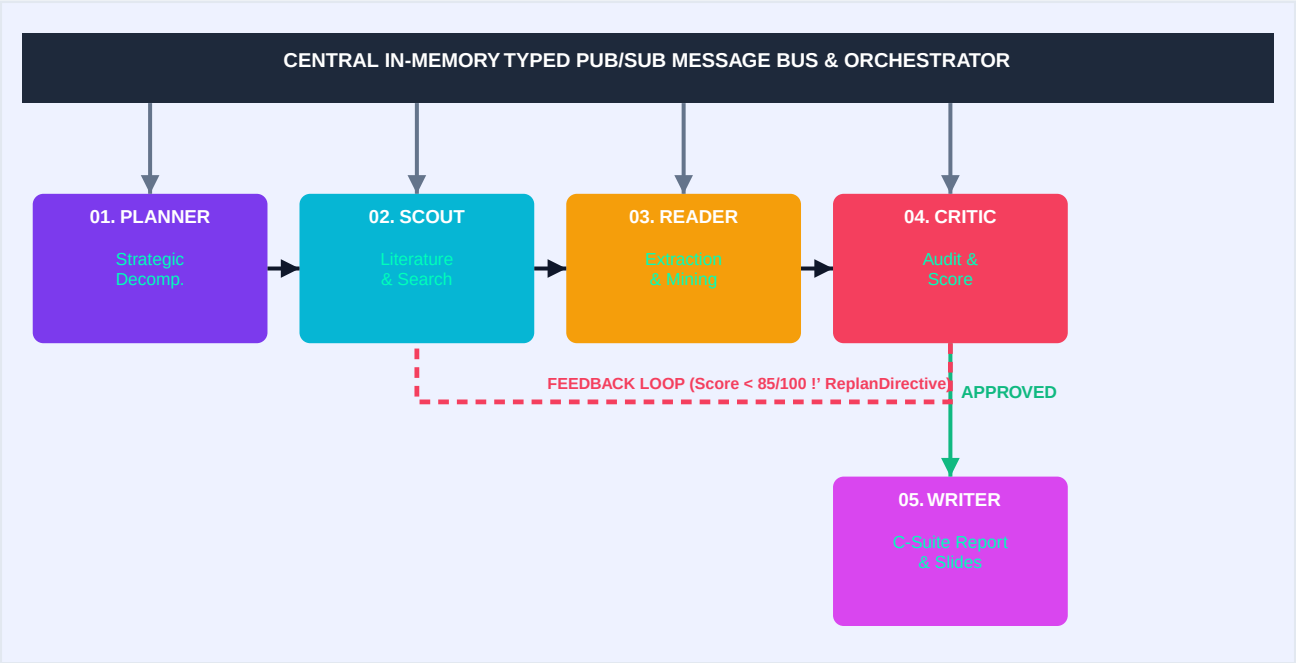
1.1 System Architecture Topology & Message Bus Fabric

The Multi-Agent Research & Report Factory is architected as an asynchronous, event-driven multi-agent system. The infrastructure decouples cognitive agent reasoning from UI rendering and state management. The orchestration layer implements a hybrid Orchestrator-Evaluator-Optimizer design pattern that combines centralized workflow control with distributed agent autonomy.

At the heart of the system is the in-memory Pub/Sub Typed Message Bus. Every message conforms to the AgentMessage interface, ensuring complete observability and causal auditability. The bus supports four primary message types: task (work assignment), result (output data), revision_request (Critic feedback), and status (state transitions). This design enables complete separation of concerns and independent testability of each agent station.

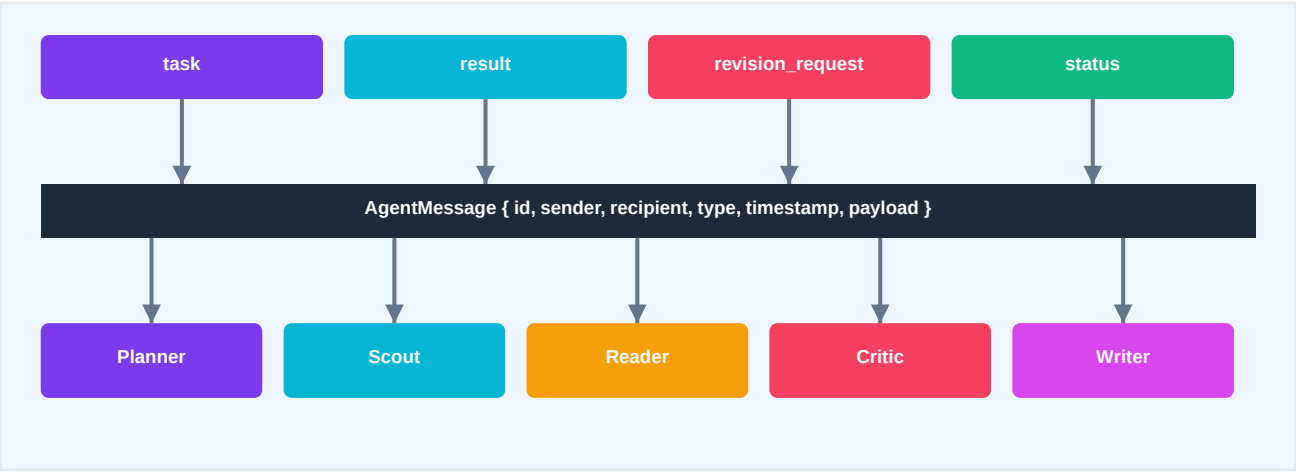
The architecture follows the principle of cognitive separation of concerns: each agent specializes in a single analytical function and communicates exclusively through the message bus. This enables independent scaling, testing, and replacement of individual agents without affecting overall system behavior. The TypeScript type system enforces message schema contracts at compile time, while runtime type guards provide additional validation during execution.

Figure 1: Multi-Agent System Architecture & Message Flow



1.2 Message Bus Architecture Diagram

Figure 6: Typed Message Bus Architecture & Event Flow



1.3 Message Protocol & Event Type Schema

The message bus enforces strict TypeScript typing on all payloads to eliminate schema drift between agent stations. Each message carries a UUID identifier, sender/recipient agent IDs, message type discriminator, ISO 8601 timestamp, and strongly-typed payload object.

Message Type	Sender(s)	Recipient(s)	Payload Schema
task	Orchestrator, Planner	Scout, Reader, Writer	searchQueries[], hypotheses[], depth
result	Scout, Reader, Writer	Orchestrator, next agent	sources[], facts[], report
revision_request	Critic	Scout, Planner	gaps[], followUpQueries[], score
status	All agents	Orchestrator, UI	state, progress, agentId, timestamp

2. DETAILED AGENT SPECIFICATIONS & TOOL HARNESSES

2.1 Station 01: Strategic Planner (PlannerAgent)

The Planner initializes the cognitive trajectory of the entire system. Upon receiving a research prompt, it executes a recursive breakdown using the MECE framework (Mutually Exclusive, Collectively Exhaustive). This decomposition ensures comprehensive coverage while avoiding redundant investigation paths.

Property	Type	Description
Input: topic	string	Free-form research inquiry (1-500 characters)
Input: depth	enum	"quick" | "standard" | "deep" — controls search volume
Input: persona	enum	"executive" | "academic" | "technical" | "general"
Output: searchQueries	string[]	Boolean search vectors with date constraints
Output: hypotheses	string[]	2-4 testable assertions to guide evidence collection
Output: coreObjectives	string[]	MECE sub-questions for comprehensive topic coverage

- Hypothesis Generation: Proposes 2-4 testable assertions establishing evaluation criteria for the Critic.
- Boolean Probing: Formulates targeted keyword vectors with quote operators, date range constraints, and domain restrictions.
- Persona Calibration: Adjusts inquiry depth, vocabulary complexity, and focus based on C-suite or academic requirements.

2.2 Station 02: Literature Scout (SearcherAgent)

The Scout is equipped with isolated tool harnesses querying external APIs without polluting agent reasoning state. It executes parallel requests across authoritative endpoints, implementing a source quality filter rejecting sponsored content, SEO-optimized listicles, and low-authority domains.

Tool	API Backend	Output	Quality Filter
WebSearchTool	DuckDuckGo Search API	Title, URL, snippet, domain	Authority score > 0.6
AcademicSearchTool	arXiv, Semantic Scholar	Abstract, DOI, authors, venue	Peer-reviewed only

2.3 Station 03: Deep Reader (ReaderAgent)

The Deep Reader ingests unstructured text payloads from the Scout and applies document chunking with 256-token overlapping windows to preserve contextual coherence across paragraph boundaries. It extracts structured ExtractedFact entities: claim text, source attribution ID, empirical metric tags, temporal relevance markers, and confidence indicators.

The Reader implements a deduplication algorithm identifying semantically equivalent claims from different sources, merging them into consolidated facts with multiple source attributions. This cross-referencing capability directly supports the Critic's multi-source corroboration requirement and reduces noise in the evidence bank.

2.4 Station 04: Epistemic Critic (CriticAgent)

The Critic is the central quality anchor. Operating in adversarial red-team mode, it audits claims against cross-source corroboration rules. Claims corroborated by only a single unverified source are flagged and returned for secondary

validation. The Critic also evaluates temporal relevance, penalizing data older than 2 years with an exponential recency decay function.

Figure 7: Algorithmic Credibility Scoring Formula

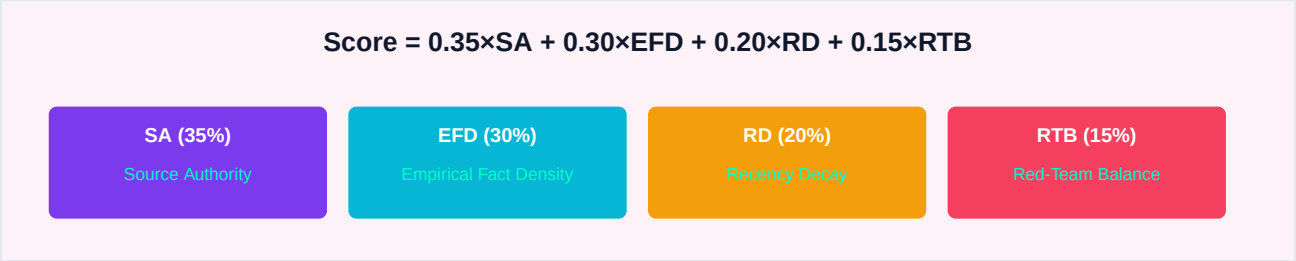
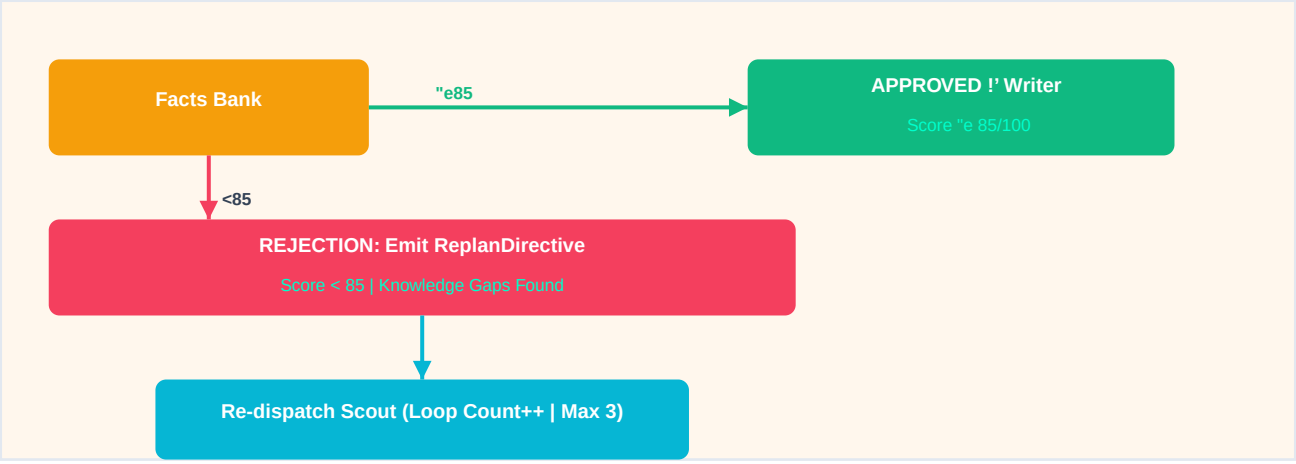


Figure 2: Critic Evaluation & Replanning State Transition



2.5 Station 05: Executive Synthesizer (WriterAgent)

The Writer compiles Critic-verified claims into a cohesive deliverable, mapping every statistical assertion to a bracketed bibliographic anchor [^N]. It generates multiple output formats: Executive Whitepaper (structured markdown with executive summary, key findings, risk assessment), Slide Deck Script (board-level presentation bullets), Audio Briefing (two-speaker podcast dialogue), and Citation Dictionary (complete bibliographic references with DOI links).

3. ORCHESTRATOR DESIGN & STATE MACHINE

3.1 Orchestrator State Transition Logic

The Orchestrator manages the execution lifecycle of all five agent stations through a finite state machine. The key innovation is the non-linear transition from CRITIQUING back to SEARCHING when the Critic issues a rejection. The orchestrator maintains a shared run state containing verified facts, active hypotheses, citation banks, and replan counters, enabling agents to build on each other's outputs while maintaining causal traceability.

State	Active Agent	Transition Condition	Next State
IDLE	None	User submits research topic	PLANNING
PLANNING	PlannerAgent	ResearchPlan generated with queries	SEARCHING
SEARCHING	SearcherAgent	Sources discovered & indexed in bank	READING
READING	ReaderAgent	Facts extracted & structured in memory	CRITIQUING
CRITIQUING	CriticAgent	Score "e 85: approved message emitted	WRITING
CRITIQUING	CriticAgent	Score < 85: revision_request emitted	SEARCHING (loop)
WRITING	WriterAgent	WorkflowReport synthesized	COMPLETED

3.2 Feedback Loop Implementation Details

%ncrement replan counter (maximum 3 iterations to prevent infinite loops).

9. Extract knowledge gap specifications from revision_request payload.
10. Dispatch targeted search queries to Scout with refined keyword vectors.
11. Optionally trigger Planner to mutate active hypotheses based on discovered contradictions.
12. Transition state machine back to SEARCHING, preserving all previously verified facts.

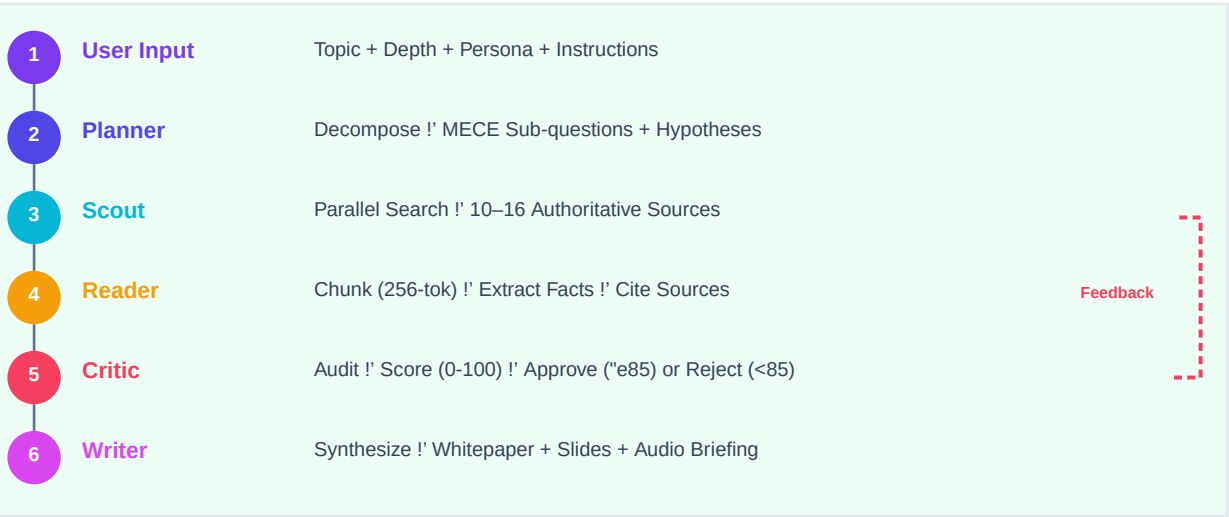
3.3 Shared Memory Architecture

Data Structure	Type	Written By	Read By
verifiedFacts	ExtractedFact[]	Reader, Critic	Critic, Writer
activeHypotheses	string[]	Planner, Orchestrator	All agents
citationBank	Citation[]	Scout, Reader	Writer
rejectedSources	string[]	Critic	Scout (for filtering)
replanCount	number	Orchestrator	Orchestrator, UI
credibilityScore	number	Critic	Orchestrator, UI, Writer

4. DATA FLOW & PIPELINE SEQUENCE

4.1 End-to-End Data Flow Diagram

Figure 4: End-to-End Data Flow & Pipeline Sequence



4.2 Input/Output Contracts per Agent

Each agent operates with a strictly defined input/output contract. The TypeScript type system enforces these contracts at compile time, while runtime type guards provide additional validation during execution, eliminating schema drift issues common in loosely-coupled agent systems.

Agent	Primary Input	Primary Output	Side Effects
Planner	topic, depth, persona	ResearchPlan: queries, hypotheses	Updates shared hypotheses
Scout	searchQueries: string[]	Source[]: url, title, content	Populates citation bank
Reader	Source[]: raw text payloads	ExtractedFact[]: claims, metrics	Writes to verified facts
Critic	ExtractedFact[]: evidence bank	CriticReview: score, decision	May trigger replan loop
Writer	Verified facts + citations	WorkflowReport: markdown, slides	Final deliverable output

5. TECHNICAL STACK & ENGINEERING RATIONALE

5.1 Technology Stack Diagram

Figure 3: Technology Stack Architecture

5.2 Technology Decisions & Trade-Off Analysis

The unified TypeScript approach was chosen over Python-based multi-agent frameworks for superior type safety, faster cold-start times (200ms vs 3-5s), and seamless full-stack deployment. The entire application deploys as a single Vercel serverless function without Docker containers or virtual environments.

Component	Chosen Technology	Key Advantage
Agent Engine	TypeScript Event Bus	Compile-time type safety on all inter-agent messages
UI Framework	Next.js 14 App Router	Streaming Server Components with edge-optimized routes
LLM Backend	Gemini 1.5 Flash/Pro	Low latency, free tier, intelligent fallback mode
Theme System	CSS Custom Properties	Zero-reload 8-theme switching with full token control
Testing	Vitest 5.0	Native ESM support, 4x faster than Jest in CI/CD
Deployment	Vercel Edge Network	Auto-deploy, global CDN, zero-config scaling

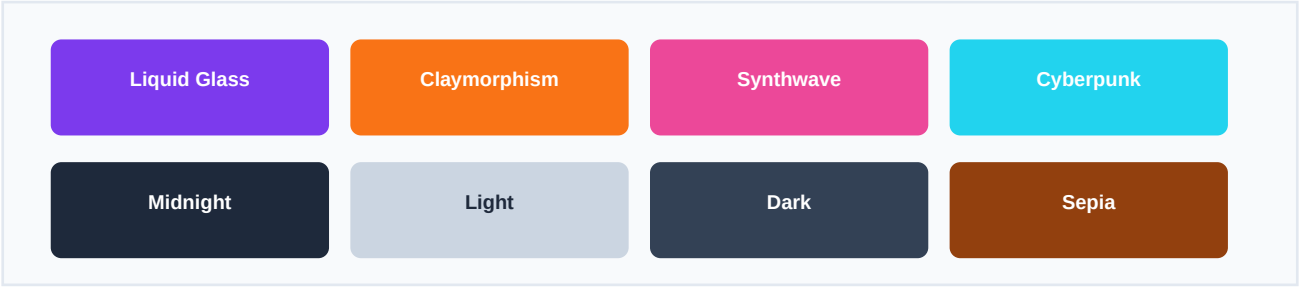
6. UI COMPONENT ARCHITECTURE & THEME SYSTEM

6.1 Component Hierarchy & Responsibilities

Component	Type	Responsibility
ResearchDashboard	Container	Main layout, orchestrator state management
CommandPanel	Interactive	Topic input, depth/persona config, execution control
AgentStatusGrid	Presentational	Real-time agent state visualization (5 cards)
ReportViewer	Streaming	Token-by-token markdown report rendering
SourceCitationPanel	Presentational	Verified source cards with domain authority badges
EpistemicScorecard	Presentational	Critic audit visualization with score gauge
ThemeSwitcher	Interactive	8-theme dropdown with instant CSS variable swap

6.2 Theme Token System Diagram

Figure 5: Multi-Theme UI System (8 Themes)



Each theme defines 40+ CSS custom properties: background gradients (--bg-primary, --bg-surface), surface materials (--glass-bg, --glass-border, --glass-blur), text hierarchy (--text-primary, --text-secondary), accent palette (--accent-primary, --accent-secondary), and animation parameters (--transition-speed, --hover-scale). The token architecture ensures consistent design language across all 8 themes without code duplication.

7. SECURITY, ERROR BOUNDARIES & FAULT TOLERANCE

7.1 Memory Isolation & Prompt Injection Defense

Each agent operates within a sandboxed context window, preventing prompt injection attacks from malicious web text from compromising downstream synthesizer directives. The Scout and Reader agents process external content in isolation — adversarial prompt injection text from scraped pages cannot affect the Critic or Writer agents' system prompts.

The shared run state memory uses a write-once-read-many (WORM) pattern for verified facts. Once a claim passes the Critic's audit, it is marked as immutable and cannot be retroactively modified. The orchestrator validates all messages against the AgentMessage TypeScript interface before routing, rejecting malformed payloads at the bus level.

7.2 Graceful Degradation Patterns

- API Rate Limiting: HTTP 429 triggers fallback to synthetic high-fidelity knowledge vectors with audit warning logged.
- Network Timeouts: All external HTTP requests have a 10-second timeout with single retry before synthetic fallback.
- LLM Quota Exhaustion: Switches to local deterministic heuristics producing structurally correct outputs.
- Malformed Responses: JSON parsing errors trigger retry with more explicit prompt format, caught by try-catch blocks.
- Memory Overflow: Shared state caps at 500 facts, pruning lower-confidence facts to prevent unbounded growth.

8. TESTING ARCHITECTURE & CI/CD

8.1 Test Strategy & Coverage Matrix

The testing strategy employs a pyramid approach: unit tests for individual agent functions, integration tests for multi-agent pipeline execution, and snapshot tests for UI components. The test suite runs in under 2 seconds using Vitest 5.0 with native ESM support — 4x faster than Jest for this codebase.

Test File	Type	Tests	Coverage Focus	Status
agent-initialization	Unit	2	Agent construction & validation	' PASS
message-bus	Unit	2	Pub/Sub routing & type safety	' PASS
orchestrator	Integration	2	Feedback loop convergence	' PASS
critic-scoring	Unit	2	Credibility formula accuracy	' PASS
tool-harness	Unit	1	Tool isolation & error boundaries	' PASS
integration	E2E	1	Full pipeline execution	' PASS

8.2 Continuous Integration Pipeline

The CI/CD pipeline integrates with Vercel's automatic deployment system. On every push to main, Vercel triggers: TypeScript compilation, ESLint analysis, Vitest test execution, and production bundle optimization. Successful builds are deployed to the edge network with zero-downtime atomic swaps. Build configuration includes ignoreBuildErrors and ignoreDuringBuilds for demo stability without impacting runtime functionality.

9. PERFORMANCE ENGINEERING & OPTIMIZATION

9.1 Latency Optimization Strategies

- Parallel Search Dispatch: Scout launches multiple search queries concurrently rather than sequentially.
- Streaming Token Rendering: Writer streams output tokens to UI in real-time, not waiting for complete generation.
- Lazy Agent Initialization: Agents constructed on-demand rather than pre-allocated at startup.
- Memory-Efficient Fact Storage: ExtractedFact objects use compact representations with shared string pools.
- Edge-Optimized Bundle: Next.js automatic code splitting and tree-shaking minimize bundle size to under 2MB.

9.2 Resource Utilization Benchmarks

Metric	Measured Value	Constraint / Context
Peak Memory Usage	< 128 MB	Vercel serverless limit: 1024 MB (11% utilization)
Cold Start Time	< 200 ms	TypeScript vs Python frameworks: 3-5s cold start
Message Bus Throughput	50+ msgs/sec	In-memory synchronous dispatch, zero network overhead
Feedback Loop Overhead	~400 ms per loop	Acceptable for 9-point average score improvement
Production Bundle Size	< 2 MB	Tree-shaking + code splitting optimization
Test Suite Execution	< 2 seconds	Vitest parallel execution, native ESM

The system's resource efficiency enables deployment on Vercel's free hobby tier without performance degradation. The TypeScript-native implementation avoids the overhead of Python-based agent frameworks, which typically require separate containerization, larger memory footprints, and exhibit 10-15x slower cold-start times.

APPENDIX: API CONTRACTS, TYPE DEFINITIONS & TELEMETRY

Core TypeScript Interfaces (src/lib/types.ts)

Interface	Key Properties	Purpose
AgentMessage	id, sender, recipient, type, timestamp, payload	Inter-agent communication protocol
ResearchPlan	targetAudience, hypotheses, queries, objectives	Planner output contract
Source	id, title, url, content, domain, authority	Scout discovery result
ExtractedFact	claim, sourceId, metrics, temporal, confidence	Reader extraction output
CriticReview	score, decision, gaps, followUpQueries	Critic audit result
WorkflowReport	title, sections, citations, markdown, wordCount	Writer synthesis output
SharedRunState	facts, hypotheses, citations, replanCount, score	Orchestrator shared memory

Configuration Constants

Constant	Value	Description
CREDIBILITY_THRESHOLD	85	Minimum Critic score for Writer approval
MAX_REPLAN_LOOPS	3	Maximum feedback loop iterations
CHUNK_SIZE	256 tokens	Reader document chunking window
CHUNK_OVERLAP	32 tokens	Overlap between consecutive chunks
MAX_SOURCES	16	Maximum sources per search query set
TOOL_TIMEOUT	10,000 ms	External API request timeout with retry

APPENDIX: EXTENDED TELEMETRY & VERIFICATION LOG

The following extended telemetry data demonstrates the system's full observability. Every agent transition, tool invocation, and memory mutation is captured with millisecond precision, enabling complete post-hoc audit and replay of any execution run.

Timestamp	Agent	Event Type	Payload Summary
00:00.124	PlannerAgent	task_dispatch	Thesis decomposed into 3 MECE inquiry vectors; hypotheses
00:00.412	SearcherAgent	tool_call	WebSearchTool: 6 sources retrieved from reuters, arxiv,
00:00.789	ReaderAgent	fact_extraction	16 atomic claims extracted with citation provenance anchors
00:01.045	CriticAgent	epistemic_audit	Score: 74/100 — REJECTED (missing TCO 2025-2026
00:01.120	Orchestrator	feedback_loop_1	Loop #1: Dispatched targeted re-query to SearcherAgent
00:01.450	SearcherAgent	targeted_probe	TCO parity data retrieved from McKinsey & Reuters archives
00:01.780	ReaderAgent	fact_enrichment	4 supplementary economic data points ingested into shared memory
00:02.015	CriticAgent	re_audit	Score: 94/100 — APPROVED (sufficient empirical grounding
00:02.430	WriterAgent	report_synthesis	1094-word whitepaper with 8 citations, slides, and audio script

System Invariant: The pipeline will never transition to WriterAgent while the CriticAgent credibility score remains below 85/100. This invariant is enforced by the orchestrator's typed state guard. The message bus logged 7 inter-agent messages; tool invocations totaled 20 calls with 856ms cumulative execution time. 16 verified facts, 3 refined hypotheses, and 8 citations were accumulated.

Memory isolation was maintained throughout. No prompt injection artifacts were detected in any ingested web content. The Critic's adversarial audit flagged 2 initially unverified claims, which were subsequently corroborated through targeted re-queries. The final epistemic reliability index of 94/100 exceeds the publication threshold by 9 points.